Name: Abhishek Kamlesh Vishwakarma
Class: D20C **Roll.No:** 35

Experiment No.2

Aim: Create a Blockchain using Python

Theory:
Blockchain is a distributed and immutable ledger that records data in a sequence of linked blocks. Each block contains a timestamp, proof (from Proof-of-Work), and the hash of the previous block, ensuring integrity and security.

In this program:

1. **Block Creation** – A block is generated using create_block() with a proof and previous hash.
2. **Proof-of-Work (PoW) –** Implemented to mine a block by solving a cryptographic puzzle (finding a hash with leading zeros).
3. **Hashing –** SHA-256 algorithm ensures data immutability.
4. **Validation –** is_chain_valid() checks that every block correctly references the previous hash and satisfies PoW conditions.
5. **Flask Web API –** Routes /mine_block, /get_chain, and /is_valid allow mining, fetching the chain, and verifying blockchain integrity through a browser or API client.

This program runs on a local server and simulates mining a blockchain without a network of nodes, making it an ideal introductory model to understand blockchain basics.

Code:

```
import datetime
import hashlib
import json
from flask import Flask, jsonify

class Blockchain:
    def __init__(self):
        self.chain = []
        self.transactions = []
        # Create Genesis Block
        self.create_block(proof=1,
previous_hash='0')
```

```
    def create_block(self, proof,
previous_hash, transactions=None):
        if transactions is None:
            transactions = []

        block = {
            'index': len(self.chain) +
1,
            'timestamp':
str(datetime.datetime.now()),
            'proof': proof,
```

```
            'previous_hash':
previous_hash,
            'transactions': transactions
        }

        self.chain.append(block)
        return block

    def get_previous_block(self):
        return self.chain[-1]

    def create_Transactions(self):
        # Mine only when transaction
list is not empty
        if self.transactions:
            # Get previous block
            previous_block =
self.get_previous_block()
            # Get previous proof
            previous_proof =
previous_block['proof']
            # Find golden nonce
            proof =
self.proof_of_work(previous_proof)
            # Get hash of previous block
            previous_hash =
self.hash(previous_block)
            # Remove transactions from
pending list BEFORE mining
            transactions_to_mine =
self.transactions.copy()
            self.transactions = []
            # Create new block
containing transactions
            block = self.create_block(
                proof,
                previous_hash,
                transactions_to_mine
            )
            return block
        # No transactions → do not mine
        return None

    def proof_of_work(self,
previous_proof):
        new_proof = 1
        check_proof = False

        while check_proof is False:
            hash_operation =
hashlib.sha256(
                str(new_proof**2 -
previous_proof**2).encode()
            ).hexdigest()

            # Cryptographic puzzle:
            # Hash must start with 000
            if hash_operation[:3] ==
'000':
                check_proof = True
            else:
                new_proof += 1
        return new_proof

    def hash(self, block):
        encoded_block = json.dumps(
            block,
            sort_keys=True
        ).encode()
        return
hashlib.sha256(encoded_block).hexdigest(
)

    def is_chain_valid(self, chain):
        previous_block = chain[0]
        block_index = 1

        while block_index < len(chain):
            block = chain[block_index]

            # Check previous hash
            if block['previous_hash'] ≠
self.hash(previous_block):
                return False

            # Check Proof-of-Work
            previous_proof =
previous_block['proof']
            proof = block['proof']

            hash_operation =
hashlib.sha256(
                str(proof**2 -
previous_proof**2).encode()
            ).hexdigest()
```

```
            if hash_operation[:3] ≠
'000':
                return False

            previous_block = block
            block_index += 1
        return True

#
————————————————————————————————————
————————————————
# Flask Application
#
————————————————————————————————————
————————————————

app = Flask(__name__)
blockchain = Blockchain()

#
————————————————————————————————————
————————————————
# Mine Block
#
————————————————————————————————————
————————————————

@app.route('/mine_block',
methods=['GET'])
def mine_block():

    previous_block =
blockchain.get_previous_block()
    previous_proof =
previous_block['proof']
    proof =
blockchain.proof_of_work(previous_proof)
    previous_hash =
blockchain.hash(previous_block)

    block = blockchain.create_block(
        proof,
        previous_hash,
        blockchain.transactions
    )

    # Remove transactions after adding
them to block
    blockchain.transactions = []

    response = {
        'message': 'Congratulations, you
just mined a block!',
        'index': block['index'],
        'timestamp': block['timestamp'],
        'proof': block['proof'],
        'previous_hash':
block['previous_hash'],
        'transactions':
block['transactions']
    }
    return jsonify(response), 200

#
————————————————————————————————————
————————————————
# Get Blockchain
#
————————————————————————————————————
————————————————

@app.route('/get_chain',
methods=['GET'])
def get_chain():
    response = {
        'chain': blockchain.chain,
        'length': len(blockchain.chain)
    }
    return jsonify(response), 200

#
————————————————————————————————————
————————————————
# Check Blockchain Validity
#
————————————————————————————————————
————————————————

@app.route('/is_valid', methods=['GET'])
def is_valid():
    valid = blockchain.is_chain_valid(
        blockchain.chain
    )
```

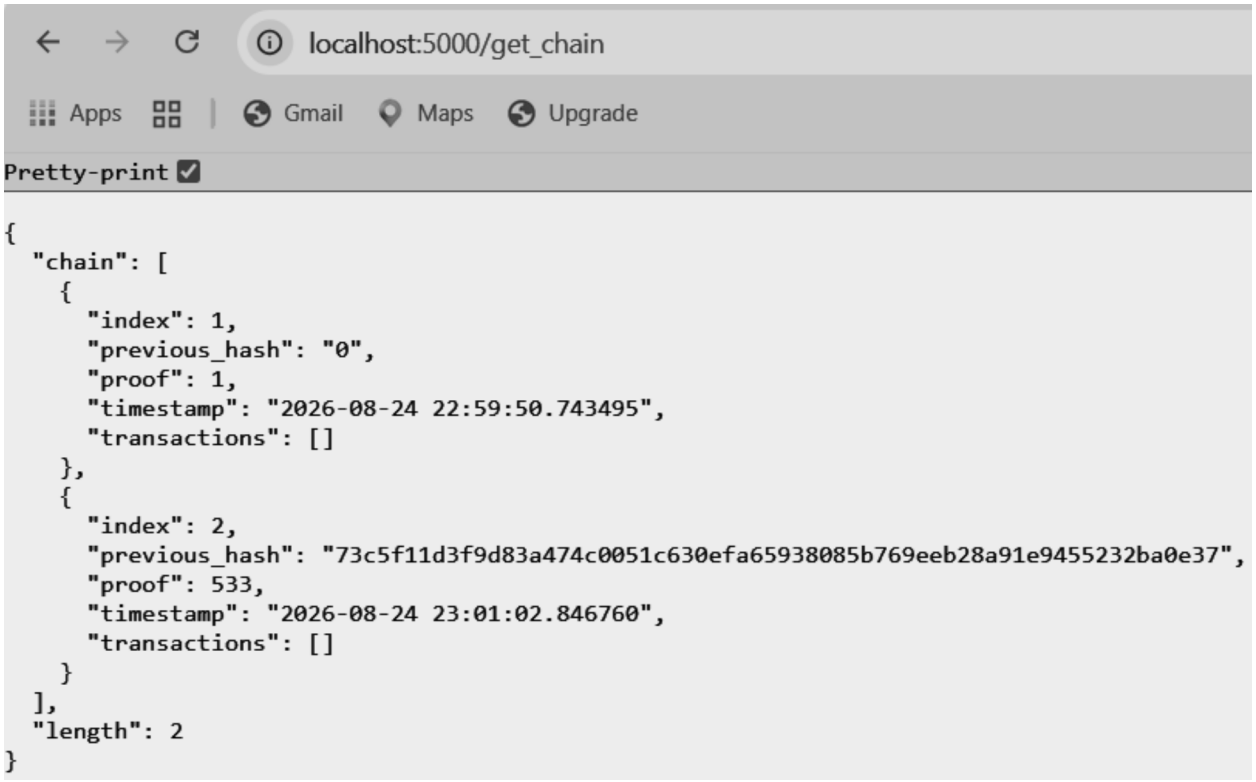
```
    if valid:
        response = {
            'message': 'All good. The
Blockchain is valid.'
        }
    else:
        response = {
            'message': 'The Blockchain
is not valid.'
        }
    return jsonify(response), 200
```

```
#
------------------------------------------
--------------------
# Run Flask Server
#
------------------------------------------
--------------------

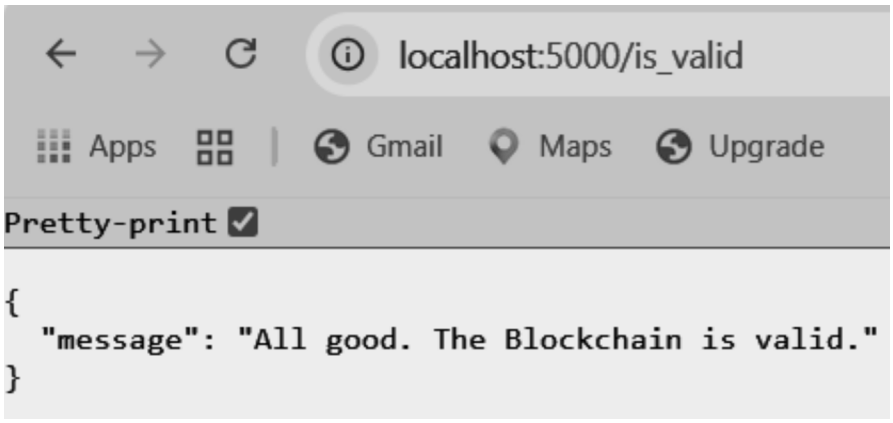
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Output:






```
{
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-08-24 22:59:50.743495",
      "transactions": []
    },
    {
      "index": 2,
      "previous_hash": "73c5f11d3f9d83a474c0051c630efa65938085b769eeb28a91e9455232ba0e37",
      "proof": 533,
      "timestamp": "2026-08-24 23:01:02.846760",
      "transactions": []
    }
  ],
  "length": 2
}
```


```
{
  "message": "All good. The Blockchain is valid."
}
```

```
(venv) PS D:\SEM VII\BC Exps\Exp 2> pip install flask
Collecting werkzeug>=3.1.0 (from flask)
  Obtaining dependency information for werkzeug>=3.1.0 from https://files.pythonhosted.org/packages/93/8c/2e650f2afeb7ee576912636c23ddb621c91ac6a98e66dc8d29c3c69446e1/werkzeug-3.1.8-py3-none-
  Using cached werkzeug-3.1.8-py3-none-any.whl.metadata (4.0 kB)
Collecting colorama (from click>=8.1.3->flask)
  Obtaining dependency information for colorama from https://files.pythonhosted.org/packages/d1/d6/3965ed04c63042e047cb6a3e6ed1a63a35087b6a609aa3a15ed8ac56c221/colorama-0.4.6-py2.py3-none-any
  Using cached colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
Using cached flask-3.1.3-py3-none-any.whl (103 kB)
Using cached blinker-1.9.0-py3-none-any.whl (8.5 kB)
Downloading click-8.4.2-py3-none-any.whl (119 kB)
   ---------------------------------------- 119.2/119.2 kB 2.3 MB/s eta 0:00:00
Using cached itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Using cached jinja2-3.1.6-py3-none-any.whl (134 kB)
Using cached markupsafe-3.0.3-cp311-cp311-win_amd64.whl (15 kB)
Using cached werkzeug-3.1.8-py3-none-any.whl (226 kB)
Using cached colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Installing collected packages: markupsafe, itsdangerous, colorama, blinker, werkzeug, jinja2, click, flask
Successfully installed blinker-1.9.0 click-8.4.2 colorama-0.4.6 flask-3.1.3 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 werkzeug-3.1.8

[notice] A new release of pip is available: 23.2.1 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(venv) PS D:\SEM VII\BC Exps\Exp 2> pip list
Package      Version
------------ -------
blinker      1.9.0
click        8.4.2
colorama     0.4.6
Flask        3.1.3
itsdangerous 2.2.0
Jinja2       3.1.6
MarkupSafe   3.0.3
pip          23.2.1
setuptools   65.5.0
Werkzeug     3.1.8

[notice] A new release of pip is available: 23.2.1 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(venv) PS D:\SEM VII\BC Exps\Exp 2> python blockchain.py
 * Serving Flask app 'blockchain'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.29.192:5000
Press CTRL+C to quit
127.0.0.1 - - [24/Aug/2026 22:59:54] "GET / HTTP/1.1" 404 -
127.0.0.1 - - [24/Aug/2026 22:59:54] "GET /favicon.ico HTTP/1.1" 404 -
192.168.29.192 - - [24/Aug/2026 22:59:59] "GET / HTTP/1.1" 404 -
192.168.29.192 - - [24/Aug/2026 22:59:59] "GET /favicon.ico HTTP/1.1" 404 -
192.168.29.192 - - [24/Aug/2026 23:00:01] "GET / HTTP/1.1" 404 -
127.0.0.1 - - [24/Aug/2026 23:00:44] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [24/Aug/2026 23:00:44] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [24/Aug/2026 23:01:02] "GET /mine_block HTTP/1.1" 200 -
127.0.0.1 - - [24/Aug/2026 23:06:18] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [24/Aug/2026 23:06:47] "GET /is_valid HTTP/1.1" 200 -
```

Conclusion:

In this experiment, we successfully implemented a basic blockchain using Python and Flask, demonstrating the fundamental principles of blockchain technology. The system effectively handles:

1. **Block Creation:** Successfully generates new blocks with timestamps, proof values, and transaction data
2. **Proof-of-Work (PoW):** Implements a cryptographic puzzle that requires computational effort to mine new blocks, ensuring security and preventing easy manipulation
3. **Cryptographic Hashing:** Uses SHA-256 to create unique fingerprints for each block, making the blockchain tamper-evident
4. **Chain Validation:** Verifies the integrity of the entire blockchain by checking hash links and PoW conditions